

nginx JS — Dynamic Reconfiguration Guide (for all workers)

Table of Contents

1. Overview	2
1.1. The multi-process problem	2
1.2. What this guide covers	2
2. suspendAllWorkers() and resumeAllWorkers()	3
2.1. The atomicity problem	3
2.2. How they work	3
2.3. Flow diagram	3
2.4. Example: atomic batch update across all workers	4
2.5. Example: atomic cross-worker rollback	5
2.6. Per-worker atomicity (suspendAcceptance)	5
2.7. API reference	6
3. Strategy 1: SharedWorker-based config broadcast	6
3.1. Architecture	6
3.2. SharedWorker setup	6
3.3. Request handler: propagate a weight change	7
3.4. Flow diagram: broadcast timeline	8
3.5. Snapshot + broadcast: rollback across all workers	8
4. Strategy 2: SharedArrayBuffer (SAB) for numeric parameters	9
4.1. When to use SAB	9
4.2. SAB-backed weight array	9
4.3. Periodic sync trigger	10
4.4. Atomic compare-and-swap for safe weight update	10
5. Strategy 3: Periodic reconciliation	11
5.1. Pattern	11
6. Caveats and best practices	12
6.1. Eventual consistency	12
6.2. In-flight requests during a config change	13
6.3. Worker restart behaviour	13
6.4. Ordering guarantees	13
6.5. Config-phase mutations (init_conf)	13



DRAFT FOR COMMENTS — This document is a work in progress. Content, API

names, and examples are subject to change before release.

1. Overview

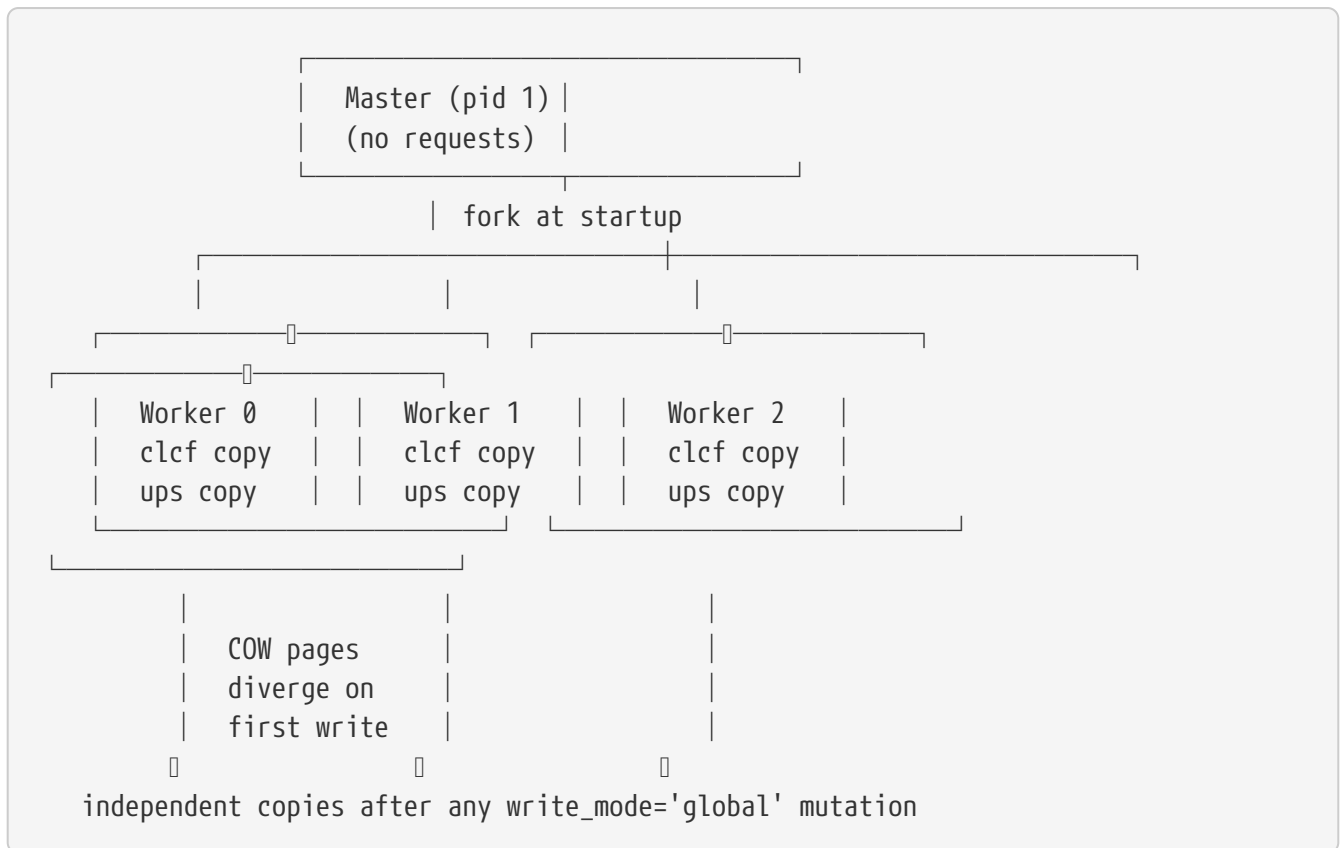
The previous guide (*for single worker*) explained `write_mode`, `read_mode`, `snapshot()`, and `restore()` — all of which operate **within a single nginx worker process**.

This guide addresses the harder problem: propagating a config change to **all workers simultaneously**.

Prerequisites: read the *for single worker* guide first.

1.1. The multi-process problem

nginx runs as a master process plus N worker processes. Each worker is a separate OS process with its own address space. After the initial fork, the workers share *no memory* — not even memory marked `write_mode='global'`.



A `write_mode='global'` assignment in a request handler in Worker 0 changes Worker 0's copy of the C struct. Workers 1 and 2 are unaffected.

1.2. What this guide covers

Three propagation strategies:

Strategy	How it works	Best for
SharedWorker broadcast	One long-lived pthread acts as the config authority. Workers push updates to it; it fans out to all workers.	Rich config objects, arbitrary JS values, ordered delivery.
SharedArrayBuffer (SAB)	A region of memory shared across all worker processes via <code>mmap</code> . Workers write/read with <code>Atomics</code> for synchronisation.	Numeric parameters (weights, timeouts), high-frequency updates.
Periodic reconciliation	Each worker polls the SharedWorker for the desired config state and applies any diff on each poll tick.	Eventual consistency is acceptable; simpler code.

2. suspendAllWorkers() and resumeAllWorkers()

2.1. The atomicity problem

The broadcast strategies in this guide deliver *eventual* consistency: there is a window between when Worker 0 applies a change and when Worker N applies the fan-out. During that window different workers may have different configs.

For changes that must be visible consistently across all workers before any new request is accepted, use `nginx.suspendAllWorkers()` and `nginx.resumeAllWorkers()`.

2.2. How they work

`await nginx.suspendAllWorkers()` sends a command to the nginx master process. The master broadcasts "suspend acceptance" to every worker and waits for acknowledgements (up to a 500 ms timeout). The Promise resolves when **all** workers have disabled their accept events.

`await nginx.resumeAllWorkers()` sends the reverse command. All workers re-enable accept events.

Both calls are **async** and return `Promise<undefined>`. The handler must be declared **async** and must `await` both calls. Always use `try/finally` to guarantee `resumeAllWorkers()` is called even on error.



`suspendAllWorkers()` is callable only from a worker process — not from config phase or from a Worker/SharedWorker thread.

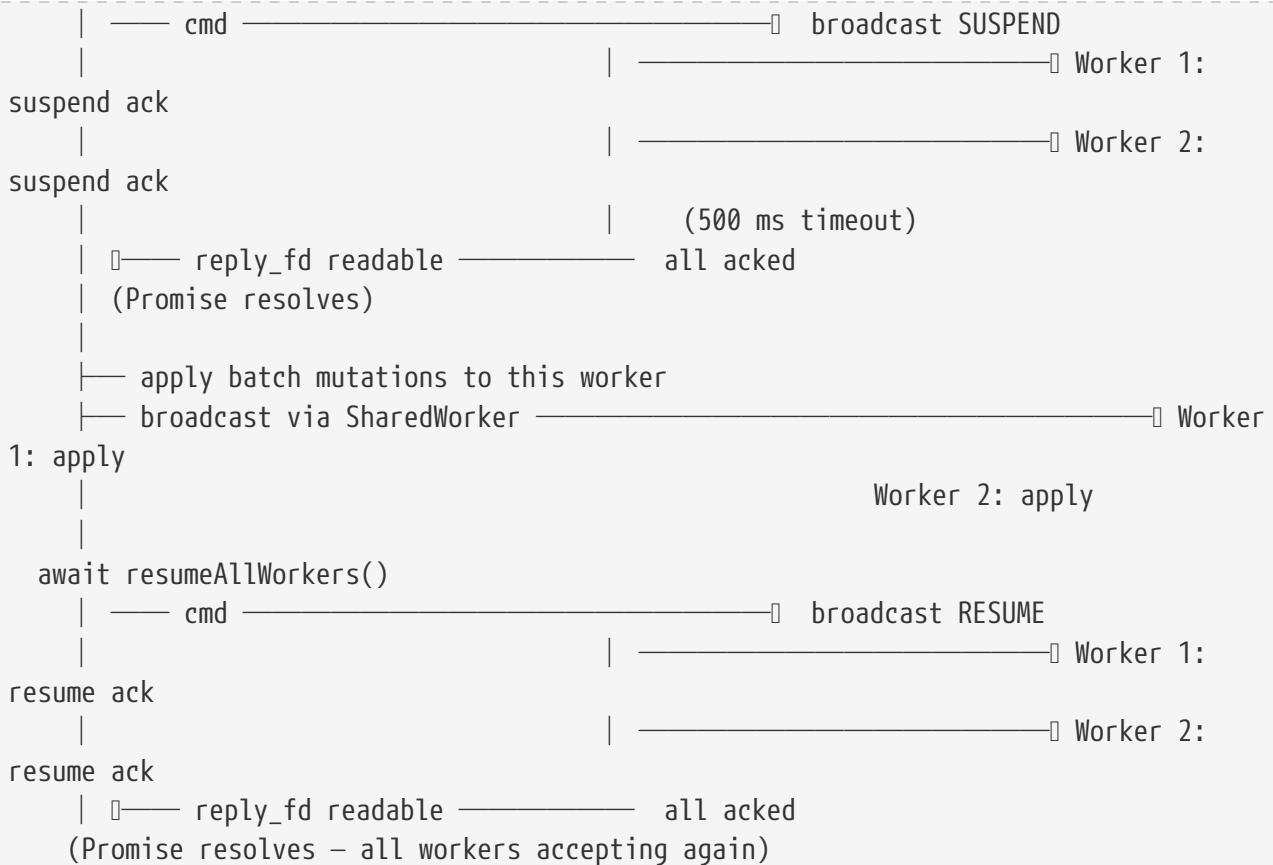
2.3. Flow diagram

Worker 0 (handler):

```
await suspendAllWorkers()
```

Master process:

Other workers:



While suspended, new TCP connections queue in the OS socket backlog (net.core.somaxconn). Keep the suspension window as short as possible — typically synchronous field writes only, not async probes.

2.4. Example: atomic batch update across all workers

```

loc.handler = async (req) => {
  const cfg = JSON.parse(await req.readBody());

  // Validate BEFORE suspending – keep the suspension window short
  const ok = await probeBackend(req);
  if (!ok) {
    req.respond(400, {}, 'validation failed');
    return;
  }

  await nginx.suspendAllWorkers();

  try {
    // Apply to this worker
    nginx.http.upstreams[0].peers.forEach((p, i) => {
      if (cfg.weights[i] !== undefined) { p.weight = cfg.weights[i]; }
    });

    // Broadcast: other workers apply in their onmessage handler
    // before their first new request after resumeAllWorkers()
  }
}

```

```

    cfgWorker.postMessage({ type: 'set', payload: cfg });

    req.json({ applied: true });
  } finally {
    await nginx.resumeAllWorkers();    // always re-enable
  }
};

```

2.5. Example: atomic cross-worker rollback

```

loc.handler = async (req) => {
  const snap = req.location.snapshot();    // save rollback state
  const cfg = JSON.parse(await req.readBody());

  // Validate before suspending
  const ok = await probeBackend(req);
  if (!ok) {
    req.respond(400, {}, 'validation failed');
    return;
  }

  await nginx.suspendAllWorkers();

  try {
    req.location.root          = cfg.root;
    req.location.proxy.readTimeout = cfg.proxy.readTimeout;

    cfgWorker.postMessage({ type: 'set', payload: cfg });
    req.json({ applied: true });

  } catch (e) {
    snap.restore();                // revert this
worker
    cfgWorker.postMessage({ type: 'rollback', payload: cfg }); // revert others
    req.respond(500, {}, e.message);

  } finally {
    await nginx.resumeAllWorkers();
  }
};

```

2.6. Per-worker atomicity (suspendAcceptance)

If only **this** worker needs an atomic batch (e.g. a local reconciliation step), use the lighter `nginx.suspendAcceptance()` / `nginx.resumeAcceptance()` pair instead. These are **synchronous**, affect only one worker, and carry zero cross-process overhead. See the *for single worker* guide for details.

2.7. API reference

```
Promise<undefined> nginx.suspendAllWorkers()  
Promise<undefined> nginx.resumeAllWorkers()
```

Both throw `TypeError` if called outside a worker process, and `InternalError` if the manager is unavailable or the reply fd cannot be registered in the event loop.

3. Strategy 1: SharedWorker-based config broadcast

3.1. Architecture

A `SharedWorker` runs in a dedicated pthread that is accessible to all workers through their per-worker channel. It holds the *desired* config state. When any worker receives a config-change request, it:

1. Validates and applies the change locally (see *for single worker* guide).
2. Posts the serialised change to the `SharedWorker`.
3. The `SharedWorker` fans out the message to all other workers.
4. Each receiving worker applies the change via `write_mode='global'`.



3.2. SharedWorker setup

Define the `SharedWorker` in `js_source`. It runs once in the master at `init_conf` and its channel is shared with every worker.

```
// js_source: config.js (runs at init_conf, before workers fork)
```

```

const cfgWorker = new SharedWorker('cfgworker.js');

cfgWorker.onmessage = (msg) => {
  // Received a fan-out from cfgWorker; apply the change to this worker
  const change = msg.data;
  applyChange(change);
};

function applyChange(change) {
  if (change.type === 'upstream.weight') {
    const ups = nginx.http.upstreams.find(u => u.name === change.upstream);
    if (!ups) { return; }
    const peer = ups.peers.find(p => p.address === change.peer);
    if (!peer) { return; }
    peer.weight = change.weight; // write_mode='global' → this worker's copy
  }
}

```

```

// cfgworker.js (runs inside the SharedWorker pthread)

// Simple in-memory desired state
const desiredWeights = {}; // { 'upstreamName/peerAddr': weight }

onmessage = (msg) => {
  const { type, payload, replyTo } = msg.data;

  if (type === 'set') {
    // Store desired state
    const key = payload.upstream + '/' + payload.peer;
    desiredWeights[key] = payload.weight;

    // Fan out to all workers (broadcast postMessage)
    postMessage({ type: 'upstream.weight', ...payload });
  } else if (type === 'get') {
    // Return current desired state to requesting worker
    postMessage({ replyTo, state: desiredWeights });
  }
};

```

3.3. Request handler: propagate a weight change

```

loc.handler = async (req) => {
  const body = JSON.parse(await req.readBody());
  const change = { upstream: body.upstream, peer: body.peer, weight: body.weight };

  // 1. Apply locally (this worker is already up-to-date after postMessage fan-out
  //    returns, but apply immediately so this request sees the new state)

```

```

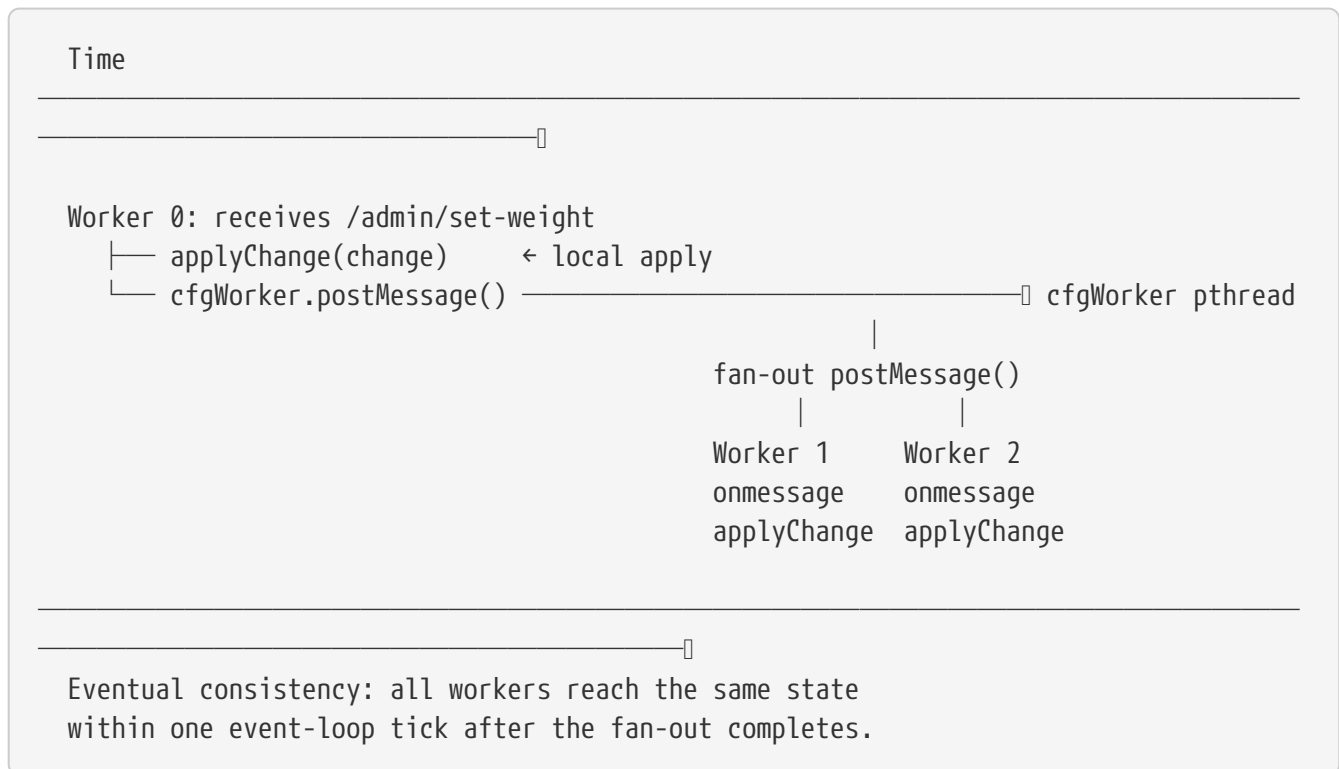
    applyChange(change);

    // 2. Tell SharedWorker to fan out to all other workers
    cfgWorker.postMessage({ type: 'set', payload: change });

    req.json({ applied: true });
  };

```

3.4. Flow diagram: broadcast timeline



3.5. Snapshot + broadcast: rollback across all workers

If validation fails after applying locally, roll back locally **and** broadcast a rollback message.

```

loc.handler = async (req) => {
  const body = JSON.parse(await req.readBody());
  const snap = req.location.snapshot(); // save current state

  try {
    applyChange(body); // apply locally
    const ok = await probeBackend(req);
    if (!ok) { throw new Error('probe failed'); }

    cfgWorker.postMessage({ type: 'set', payload: body });
    req.json({ applied: true });
  } catch (e) {
    snap.restore(); // revert local copy
  }
};

```

```

    // Broadcast rollback (SharedWorker restores original desired state)
    cfgWorker.postMessage({ type: 'rollback', payload: body });
    req.respond(500, {}, 'apply failed: ' + e.message);
  }
};

```

The SharedWorker's rollback handler restores **desiredWeights** and fans the original values back to all workers:

```

// inside cfgworker.js onmessage:
} else if (type === 'rollback') {
  const key = payload.upstream + '/' + payload.peer;
  const prev = desiredWeights[key]; // value before the failed 'set'

  if (prev !== undefined) {
    postMessage({ type: 'upstream.weight', ...payload, weight: prev });
  }
}

```



Between the **set** fan-out and the **rollback** fan-out there is a brief window where some workers may have the new value and others the old one. This is inherent to eventual-consistency approaches and is acceptable for most configuration use cases.

4. Strategy 2: SharedArrayBuffer (SAB) for numeric parameters

4.1. When to use SAB

SharedArrayBuffer provides shared memory visible to all worker processes simultaneously. Combined with **Atomics**, it gives synchronous, lock-free reads and compare-and-swap writes without any message passing.

Use SAB for high-frequency, low-latency updates of numeric parameters: upstream peer weights, timeout values, rate limits.

4.2. SAB-backed weight array

```

// js_source: sab_config.js

// Create a SAB in the master (before fork).
// Layout: one Int32 slot per upstream peer, indexed by peer position.
const PEER_COUNT = 8;

```

```

const weightBuf = new SharedArrayBuffer(PEER_COUNT * 4);
const weights = new Int32Array(weightBuf);

// Initialise from current nginx config
nginx.http.upstreams.forEach(ups => {
  ups.peers.forEach((p, i) => {
    Atomics.store(weights, i, p.weight);
  });
});

// Worker handler: read-modify-write a single peer weight
function setWeight(peerIdx, newWeight) {
  Atomics.store(weights, peerIdx, newWeight);

  // Apply to this worker's nginx struct immediately
  const peer = nginx.http.upstreams[0].peers[peerIdx];
  if (peer) { peer.weight = newWeight; }
}

// Worker handler: apply all SAB values to this worker's structs
// (call periodically or on a trigger to "sync" this worker)
function syncWeightsFromSAB() {
  nginx.http.upstreams[0].peers.forEach((p, i) => {
    const desired = Atomics.load(weights, i);
    if (p.weight !== desired) { p.weight = desired; }
  });
}

```

4.3. Periodic sync trigger

Workers cannot pro-actively receive SAB updates — they must poll. Use `nginx.setTimeout` to schedule a periodic sync:

```

// js_source: start a reconciliation loop in every worker
(async function reconcileLoop() {
  while (true) {
    await nginx.setTimeout(1000); // every 1 s
    syncWeightsFromSAB();
  }
})();

```

4.4. Atomic compare-and-swap for safe weight update

```

loc.handler = (req) => {
  const idx = parseInt(req.getVar('arg_idx'));
  const newWeight = parseInt(req.getVar('arg_weight'));
  const expected = parseInt(req.getVar('arg_expected')); // CAS guard

```

```

const actual = Atomics.compareExchange(weights, idx, expected, newWeight);

if (actual !== expected) {
  req.respond(409, {}, 'CAS failed: weight changed by another worker');
  return;
}

// CAS succeeded – apply locally; other workers will pick it up on next poll
const peer = nginx.http.upstreams[0].peers[idx];
if (peer) { peer.weight = newWeight; }

req.json({ applied: true, prev: actual });
};

```

5. Strategy 3: Periodic reconciliation

5.1. Pattern

Rather than reacting to change events, each worker independently polls a SharedWorker (the "config authority") and applies any diff.

```

// js_source: reconcile.js

const cfgAuth = new SharedWorker('cfgauth.js');
let desiredConfig = null; // cached desired state (updated by onmessage)

cfgAuth.onmessage = (msg) => {
  desiredConfig = msg.data.config;
};

// Request desired config from authority at startup
cfgAuth.postMessage({ type: 'get' });

// Reconciliation loop: every 5 s, apply diff
(async function reconcileLoop() {
  while (true) {
    await nginx.setTimeout(5000);
    if (!desiredConfig) { continue; }

    nginx.http.upstreams.forEach(ups => {
      const desired = desiredConfig.upstreams?.[ups.name];
      if (!desired) { return; }

      ups.peers.forEach((p, i) => {
        const dw = desired.peers?.[i]?.weight;
        if (dw !== undefined && p.weight !== dw) {
          p.weight = dw;
        }
      });
    });
  }
})();

```

```

    }
  });
});
}
})();

```

```

// cfgauth.js (SharedWorker – single source of truth)

let config = { upstreams: {} };

onmessage = (msg) => {
  if (msg.data.type === 'set') {
    // Deep-merge incoming change into desired config
    merge(config, msg.data.config);
    postMessage({ type: 'config', config }); // broadcast to all workers
  } else if (msg.data.type === 'get') {
    postMessage({ type: 'config', config }); // reply to requester
  }
};

function merge(target, src) {
  for (const k of Object.keys(src)) {
    if (src[k] && typeof src[k] === 'object' && !Array.isArray(src[k])) {
      target[k] = target[k] || {};
      merge(target[k], src[k]);
    } else {
      target[k] = src[k];
    }
  }
}

```

6. Caveats and best practices

6.1. Eventual consistency

All three strategies deliver **eventual** consistency. Between the moment a change is initiated and the moment all workers have applied it there is a window during which workers may disagree.

For most config-change use cases (upstream weights, timeouts, feature flags) this is acceptable. For strict consistency, use a coordinated handshake:

```

// Worker 0 waits for acknowledgement from all other workers before
// considering the change applied.
// (Implementation depends on the SharedWorker protocol design.)

```

6.2. In-flight requests during a config change

Requests that started before the config change will complete with the old config values. This is intentional — mid-request config changes are never applied retroactively.

6.3. Worker restart behaviour

When a worker crashes and is restarted by the master, the new worker forks from the master's original config. It will not automatically pick up runtime mutations applied to sibling workers unless it receives a reconciliation message from the SharedWorker or re-reads a SAB.

A healthy reconciliation loop (Strategy 3) handles this automatically: the restarted worker will catch up within one poll period.

6.4. Ordering guarantees

The SharedWorker pthread processes messages sequentially (one at a time). Fan-out is also sequential. Workers may apply fan-out messages in different orders if they happen to be busy when the message arrives — do not rely on cross-worker ordering.

6.5. Config-phase mutations (`init_conf`)

Config-phase mutations (in top-level `js_source` code, before `loc.handler` is set) run in the master before fork. They are inherited by all workers at fork time and require no broadcast. Only *request-phase* mutations need the strategies described in this guide.